Hadoop & Spark - ISEN 2025/2026

Lab: Partitioning

For this Lab you will need:

- The 3.11 Lab

As a reminder

Check 3.1 to login in NGROK, Kaggle, install Java or Spark and import libraries.

Partitioning

First let's make sure our spark session has `spark.sql.adaptive.enabled` set to false. It could ruin the purpose of this lab if you do not do it as Spark will dynamically find a right number of partitions by itself

```
spark = (
    SparkSession
      .builder
      .appName("PartitionApp")
      .master("local[*]")
      .config('spark.ui.port', '4050')
      .config("spark.dynamicAllocation.enabled", "false")
      .config("spark.sql.adaptive.enabled", "false") # <- very important !
      .enableHiveSupport()
      .getOrCreate()
)
spark.conf.set("spark.sql.repl.eagerEval.enabled", True)
sc = spark.sparkContext
spark
```

Partitioning

Check the current parallelism:

```
sc.defaultParallelism
```

Partitions: Reading data

Let's check partitions for a relatively small dataset `TaxiZones`

```
taxiZonesDF = (
                  spark
                    .read
                    .option("inferSchema", "true")
                    .csv("taxi+_zone_lookup.csv")
                )
print("Partitions = "    + str(taxiZonesDF.rdd.getNumPartitions()))
print("Record Count = "  + str(taxiZonesDF.count()))
```

Partitions: Reading data

Let's check with a bigger dataset `yellow_tripdata_2020-01.csv`

```
yellowTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .option("inferSchema", "true")
                    .csv("yellow_tripdata_2020-01.csv")
                )
print("Partitions = "    + str(yellowTaxiDF.rdd.getNumPartitions()))
print("Record Count = "  + str(yellowTaxiDF.count()))
```

Partitions: Reading data

Let's manually declare our maximum partition size at 64mb

```
spark.conf.set( "spark.sql.files.maxPartitionBytes", "64m" )
```

And check on the dataset once again

```
yellowTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .option("inferSchema", "true")
                    .csv("yellow_tripdata_2020-01.csv")
                )

print("Default Parallelism = " + str( sc.defaultParallelism))
print("Partitions = " + str( yellowTaxiDF.rdd.getNumPartitions()))
print("Record Count = " + str(yellowTaxiDF.count()))
```

Partitions: Reading data

Let's create this quick function that will show us more details on each partition of the data for a given column:

```
from pyspark.sql.functions import *

def getDataFrameStats(dataFrame, columnName):
    outputDF = (
                    dataFrame
                        # Get partition number for each record
                        .withColumn("Partition Number", spark_partition_id())
                        # Group by partition, and calculate stats for a column
                        .groupBy("Partition Number")
                        .agg(
                                 count("*").alias("Record Count"),
                                 min(columnName).alias("Min Column Value"),
                                 max(columnName).alias("Max Column Value")
                            )
                        .orderBy("Partition Number")
                )
    return outputDF
```

Partitions: Reading data

And use it on `yellowTaxiDF` as an example:

```
getDataFrameStats( yellowTaxiDF, "PULocationID" ).show()
```

Partitions: shuffle

Let's check the default number of partitions:

```
spark.conf.get("spark.sql.shuffle.partitions")
```

If you did not changed it, it should be 200

Partitions: shuffle

We will apply a shuffle operation and check DataFrame stats with the previously created function:

```
yellowTaxiGroupedDF = (
                            yellowTaxiDF
                                .groupBy("PULocationID")
                                .agg(sum("total_amount"))
                        )
print("Partitions = "  + str( yellowTaxiGroupedDF.rdd.getNumPartitions() ))
getDataFrameStats( yellowTaxiGroupedDF, "PULocationID" ).show()
```

Partitions: shuffle

Now let's fix the shuffle partitions to 3:

```
spark.conf.set( "spark.sql.shuffle.partitions", 3 )
```

And launch the same operation:

```
yellowTaxiGroupedDF = (
                            yellowTaxiDF
                                .groupBy("PULocationID")
                                .agg(sum("total_amount"))
                        )
print("Partitions = "  + str( yellowTaxiGroupedDF.rdd.getNumPartitions() ))
getDataFrameStats( yellowTaxiGroupedDF, "PULocationID" ).show()
```

Repartitions methods

Let's check the stats with our custom function:

```
getDataFrameStats( yellowTaxiDF, "PULocationID" ).show()
```

We will see the changes made to this stats using different repartitions methods

Repartitions methods: Round-Robin

This create equal-sized partitions, data is not co-located

```
repartionedDF1 = yellowTaxiDF.repartition( 14 )
getDataFrameStats( repartionedDF1, "PULocationID").show()
```

Repartitions methods: Hash partitioning

Co-locates the data & partition sizes may not be the same.

```
repartionedDF2 = yellowTaxiDF.repartition( "PULocationID" )
getDataFrameStats( repartionedDF2, "PULocationID" ).show()
```

You can define the number of partitions aswell:

```
repartionedDF3 = yellowTaxiDF.repartition( 14, "PULocationID" )
getDataFrameStats( repartionedDF3, "PULocationID" ).show()
```

Repartitions methods: Range partitioning

Sort and co-locates the data & partition sizes may not be the same

```
repartionedDF4 = yellowTaxiDF.repartitionByRange( 14, "PULocationID" )
getDataFrameStats( repartionedDF4, "PULocationID" ).show()
```

Note: you can see a pattern with Min & Max column

Repartitions methods: Coalesce

You can use this to reduce the number of partitions (remember, our previous value for yellowTaxiDF was 3)

```
coalescedDF = yellowTaxiDF.coalesce(2)
getDataFrameStats( coalescedDF, "PULocationID" ).show()
```